Documentación técnica: Arena de futbolito automatizada

1. Resumen ejecutivo

Construimos una arena de futbolito autónoma para dos jugadores, inspirada en Hyperlaps Initial, controlada por un Arduino UNO con FreeRTOS y una cámara procesada con Python y OpenCV. Cada portero se calibra solo con finales de carrera, sigue la pelota con un motor NEMA 17, la golpea con un servomotor y pierde una de sus 10 vidas, mostradas en LEDs, cuando recibe un gol.

El código está en el repositorio soffiblanco/Arena-futboliito: `arena_futbolito_freertos_completo.ino` (1014 líneas, firmware) y `camara_arduino.py` (159 líneas, visión artificial).

Requisito mínimo del enunciado	Cómo lo cumplimos
Arena física operativa	Arena de cartón construida desde cero
Dos naves con movimiento lateral motorizado	2 motores NEMA 17 con drivers TB6600, correa y poleas
Mecanismo de golpeo por jugador	2 servomotores SG90 controlados por software
Detección automática de la bola	Cámara + OpenCV (color HSV y circularidad)
Detección de anotaciones	2 sensores infrarrojos con confirmación y rearme
10 puntos de vida por jugador	20 LEDs manejados con registros 74HC595
Software con Arduino y FreeRTOS	Tarea periódica `tareaTiempoReal` con `vTaskDelayUntil`
Partida completa y reinicio	Estados apagado → calibrando → jugando → finalizado; reinicio con el interruptor

2. Análisis del problema y decisiones de diseño

El problema es de tiempo real: el portero debe ver la pelota, decidir y moverse antes de que llegue a la portería, y una respuesta correcta pero tardía equivale a un gol. Por eso dividimos el sistema en dos partes con exigencias de tiempo distintas: la percepción

pesada en una computadora y el control de actuadores, con plazos cortos y deterministas, en el Arduino.

Alcance. Modalidad mínima de dos jugadores en modo autónomo. No implementamos modo manual ni ampliación a tres o cuatro jugadores.

Restricciones. El Arduino UNO tiene 2 KB de SRAM y 32 KB de flash; los pines digitales no alcanzan para 20 LEDs, 2 motores, 2 servos y 6 sensores; el presupuesto del equipo era limitado.

Decisiones técnicas y su justificación (el enunciado permite variaciones documentadas):

Decisión	Referencia del enunciado	Justificación
Estructura de cartón	Panel rígido y paredes transparentes	Bajo costo y fácil de cortar y modificar durante las pruebas
Cámara + OpenCV en PC	Sensores ópticos, cámaras o matrices	Da la posición en X y Y con un solo sensor; el UNO no puede procesar video
Servomotores SG90 para golpear	Solenoide JF-0520B de 12 V	Se alimentan a 5 V, no requieren MOSFET ni diodo de protección y su ángulo es programable
Motores NEMA 17 + TB6600	Banda GT2 y poleas de 16 dientes	Posicionamiento preciso contando pasos, sin encoder
Finales de carrera	Estrategia de posición de referencia	Calibración automática y protección contra golpes en los extremos
Registros 74HC595	Barra LED de 10 puntos	Controlan los 20 LEDs con solo 3 pines
Sensores infrarrojos de gol	Emisor láser y receptor	Más seguros para los ojos y fáciles de montar en cartón

Riesgos identificados. Iluminación variable que afecta la detección por color; latencia del enlace cámara → PC → serial; rigidez limitada del cartón, que puede desalinear los rieles con el uso.

3. Requerimientos

Priorizamos con MoSCoW: **Debe** es indispensable para el funcionamiento autónomo mínimo, **Debería** mejora la calidad, **Podría** es una extensión.

3.1 Requerimientos funcionales

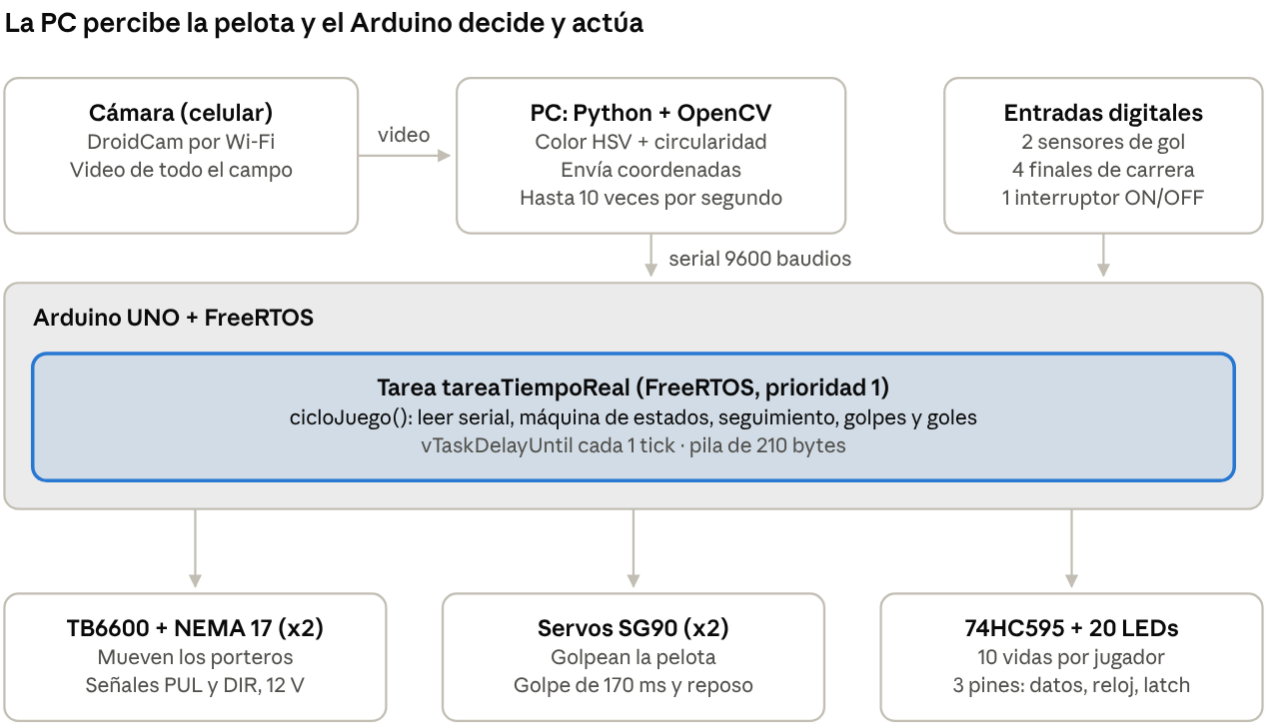
ID	Requerimiento	Prioridad	Implementación
RF-01	Encender y apagar el juego con un interruptor	Debe	Pin 7, lectura en `cicloJuego()`
RF-02	Calibrar cada portero automáticamente: izquierda, derecha y centro	Debe	`iniciarCalibracion()`, `calibrar()`
RF-03	Detectar la pelota y obtener sus coordenadas	Debe	`camara_arduino.py` (OpenCV)
RF-04	Enviar las coordenadas al Arduino y validarlas	Debe	Serial 9600 baudios; `leerSerial()`, `procesarLinea()`
RF-05	Mover cada portero hacia la posición de la pelota	Debe	`objetivoPelota()`, `seguimiento()`
RF-06	Golpear la pelota cuando entra en la zona del portero	Debe	`actualizarServos()`, `actualizarGolpe()`
RF-07	Detectar goles sin conteos dobles	Debe	`revisarGol()` con confirmación de 30 ms y rearme de 300 ms
RF-08	Restar una vida y actualizar los LEDs	Debe	`registrarGol()`, `actualizarVidas()`
RF-09	Declarar ganador y detener el juego	Debe	Estado `FINALIZADO`, `celebrar()`
RF-10	Reiniciar sin reprogramar	Debe	Interruptor OFF → ON llama a `iniciarJuego()`
RF-11	Detener actuadores ante un error	Debe	`fallo()` → estado `ERROR_SISTEMA`
RF-12	Mostrar diagnóstico por el monitor serial	Debería	Mensajes de calibración, golpes, goles y errores
RF-13	Estimar la trayectoria y velocidad de la pelota	Debería	Propuesta en la sección 10.4
RF-14	Modo manual con joystick	Podría	No implementado

3.2 Requerimientos no funcionales

ID	Requerimiento	Métrica o criterio
RNF-01	Tiempo de respuesta	Ciclo de control cada tick de FreeRTOS; datos de cámara hasta 10 por segundo; dato descartado si tiene más de 600 ms
RNF-02	Seguridad mecánica	Margen de 80 pasos en cada extremo; paro inmediato si se activa un final de carrera durante el juego
RNF-03	Seguridad eléctrica	Fuente de 12 V separada de la lógica de 5 V; tierras unidas; interruptor de apagado
RNF-04	Robustez de datos	Se descartan mensajes mal formados o fuera de rango; el portero se detiene si se pierde la pelota
RNF-05	Uso de memoria	Una sola tarea con pila de 210 bytes para caber en 2 KB de SRAM
RNF-06	Precisión	Tolerancia de 12 pasos entre posición actual y objetivo
RNF-07	Mantenibilidad	Parámetros (ángulos, tiempos, márgenes) como constantes al inicio del código
RNF-08	Confiabilidad	Límite de 30 s y 10 000 pasos por etapa de calibración para detectar fallas

4. Arquitectura general hardware–software

El sistema tiene tres niveles: entradas (cámara y sensores), un controlador (Arduino con FreeRTOS) y actuadores (motores, servos y LEDs). La visión artificial corre fuera del microcontrolador porque procesar video excede sus 2 KB de RAM.



Arquitectura hardware–software · entradas, controlador y actuadores

La computadora solo envía coordenadas; toda decisión de movimiento, golpe y conteo de vidas se toma en el Arduino, dentro de la tarea de FreeRTOS.

5. Construcción mecánica

La arena se construyó desde cero en cartón, con dos porterías enfrentadas: el portero 1 en el lado superior de la imagen de la cámara y el portero 2 en el inferior. Bajo el tablero quedan los motores, drivers y cableado, y la cámara se monta sobre la arena para ver todo el campo.

Movimiento de las naves. Cada portero va montado en un carro que se desliza sobre varillas guía. Un motor NEMA 17 mueve el carro mediante correa dentada y poleas, que convierten la rotación en desplazamiento lineal en ambos sentidos. La posición se conoce contando pasos desde la calibración, sin encoder.

Límites y referencia. Cada riel tiene dos finales de carrera, uno por extremo. Al encender, el carro busca el extremo izquierdo, luego el derecho para medir el recorrido total en pasos, y regresa a la mitad. Durante el juego, el software deja 80 pasos de margen en cada lado para no chocar con los extremos.

Mecanismo de golpeo. Cada portero lleva un servomotor SG90 que gira de su ángulo de reposo (90°) al de golpe (55° para el portero 1 y 125° para el portero 2, porque están montados en espejo) durante 170 ms y regresa. Un tiempo mínimo de 500 ms entre golpes evita activaciones repetidas.

Detección de goles. Cada portería tiene un sensor infrarrojo de obstáculos (módulo con emisor, receptor y potenciómetro de sensibilidad) que se activa cuando la pelota cruza la línea.

6. Diseño eléctrico y electrónico

El sistema usa dos fuentes separadas con tierra común: 12 V DC (al menos 5 A) para los drivers TB6600 y los motores, y 5 V (USB o fuente regulada) para el Arduino, los servos, los sensores y los 74HC595. Separar la potencia de la lógica evita que los picos de corriente de los motores reinicien el microcontrolador.

Etapa de potencia. Los TB6600 aíslan al Arduino de la corriente de los motores: el Arduino solo envía señales de pulso (PUL) y dirección (DIR) y el driver entrega la corriente a las bobinas A+/A− y B+/B−. Los pines ENA se dejan sin conectar, por lo que los drivers están siempre habilitados.

Entradas. Los finales de carrera, los sensores de gol y el interruptor usan `INPUT_PULLUP`, lo que elimina resistencias externas. Un final presionado se lee como HIGH; un sensor de gol activo y el interruptor encendido se leen como LOW.

Marcador. Los LEDs, cada uno con resistencia de 220–330 Ω, se controlan con registros 74HC595 en cascada que reciben los datos en serie por 3 pines (datos, reloj y latch).

Tabla de pines (según el código)

Pin	Señal	Componente
2	DATOS (DS)	74HC595
3	RELOJ (SHCP)	74HC595
4	LATCH (STCP)	74HC595
5	GOL1	Sensor de gol, portero 1
6	GOL2	Sensor de gol, portero 2
7	INTERRUPTOR	Interruptor ON/OFF
8	PIN_SERVO1	Servo golpeador 1
9	PIN_SERVO2	Servo golpeador 2
10	PUL1	TB6600 portero 1
11	DIR1	TB6600 portero 1
13	PUL2	TB6600 portero 2

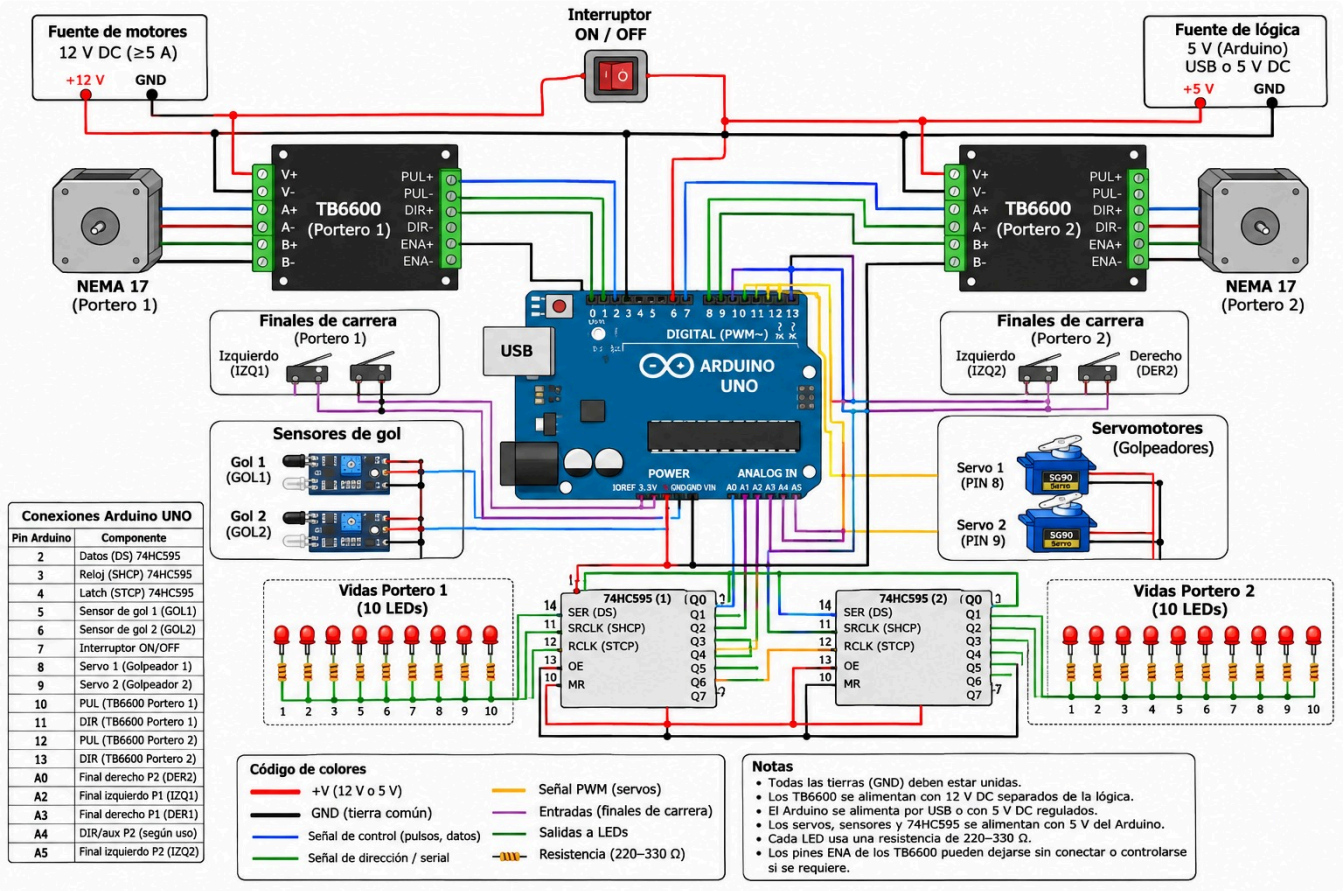
Pin	Señal	Componente
A4	DIR2	TB6600 portero 2
A2	IZQ1	Final izquierdo, portero 1
A3	DER1	Final derecho, portero 1
A5	IZQ2	Final izquierdo, portero 2
A0	DER2	Final derecho, portero 2

Observaciones sobre el diagrama (el código es la referencia):

- El diagrama asigna PUL del portero 2 al pin 12 y DIR al 13; el código usa el 13 para PUL y A4 para DIR.
- El diagrama muestra dos 74HC595 (16 salidas), pero 20 LEDs necesitan tres; además, la función `leds()` envía tres bytes, uno por registro.

Recomendación de seguridad. Dos servos SG90 alimentados desde el pin de 5 V del Arduino pueden provocar caídas de voltaje al golpear; conviene alimentarlos con una fuente de 5 V aparte, con tierra común.

Diagrama eléctrico completo del sistema:



7. Arquitectura de software

El software tiene dos programas que se comunican por serial: `camara_arduino.py` en la computadora, que percibe, y el firmware `.ino` en el Arduino, que decide y actúa. Todo el firmware es no bloqueante: ninguna función usa `delay()`; cada una guarda su estado y consulta `micros()` o `millis()` para saber si le toca actuar.

7.1 Organización del repositorio

Archivo	Lenguaje	Responsabilidad
`camara_arduino.py`	Python 3 (OpenCV, NumPy, pySerial)	Captura de video, detección de la pelota y envío de coordenadas
`arena_futbolito_freertos_completo.ino`	C++ Arduino (Servo, Arduino_FreeRTOS)	Calibración, seguimiento, golpes, goles, vidas, estados y FreeRTOS

7.2 Estructuras de datos

- `Motor`: pines, etapa de calibración, pasos contados, recorrido total, mitad, posición actual, objetivo y marcas de tiempo del último pulso. Una instancia por portero (`m1`, `m2`), así las mismas funciones sirven para ambos.
- `Golpeador`: servo, ángulos de reposo y golpe, si está golpeando, si ya golpeó en esta entrada a la zona y tiempos del último golpe (`g1`, `g2`).
- Variables de juego: `vidas1`, `vidas2`, `ganador` y las banderas de confirmación y rearme de cada sensor de gol.
- Variables de cámara: `pelotaX`, `pelotaY`, `anchoROI`, `altoROI`, `ultimoDato` y `camaraValida`.

7.3 Métodos y problema que resuelve cada uno

Método	Problema que resuelve
`leds()`, `patrones()`	Envía 3 bytes a los 74HC595 para encender cualquier combinación de los 20 LEDs con 3 pines

Método	Problema que resuelve
`actualizarVidas()`	Convierte las vidas en una barra de LEDs encendidos con `(1 << vidas) - 1`
`parar()`, `sentido()`	Detienen el pulso y cambian la dirección, con pausa de 500 ms para que el driver la registre
`preparar()`	Configura pines del motor y finales de carrera con pull-up
`paso()`	Genera un pulso del motor sin bloquear: sube y baja la señal según el tiempo transcurrido
`etapa()`, `iniciarCalibracion()`, `calibrar()`	Encuentran ambos extremos, miden el recorrido y llevan el portero al centro, con límites de tiempo y pasos
`leerSerial()`, `procesarLinea()`	Arman la línea recibida byte a byte y validan formato y rango antes de aceptar las coordenadas
`objetivoPelota()`	Convierte la X de la pelota en píxeles a una posición en pasos dentro del recorrido seguro
`seguimiento()`	Mueve el portero hacia el objetivo y lo detiene si se pierde la cámara o se activa un final
`actualizarServos()`, `actualizarGolpe()`	Deciden si la pelota está en la zona de golpe, golpean una vez por entrada y regresan el servo a reposo
`revisarGol()`, `registrarGol()`	Confirman el gol (30 ms), evitan conteos dobles (rearme de 300 ms) y restan vidas
`celebrar()`	Hace parpadear 12 veces los LEDs del ganador sin bloquear
`iniciarJuego()`, `apagar()`	Reinician vidas, sensores y calibración, o detienen todo al apagar
`fallo()`, `detenerServos()`	Llevan el sistema a estado seguro y reportan la causa por serial
`cicloJuego()`	Máquina de estados: decide qué hacer en cada estado

Método	Problema que resuelve
`tareaTiempoReal()`	Tarea de FreeRTOS que ejecuta el ciclo de forma periódica

7.4 Protocolo serial

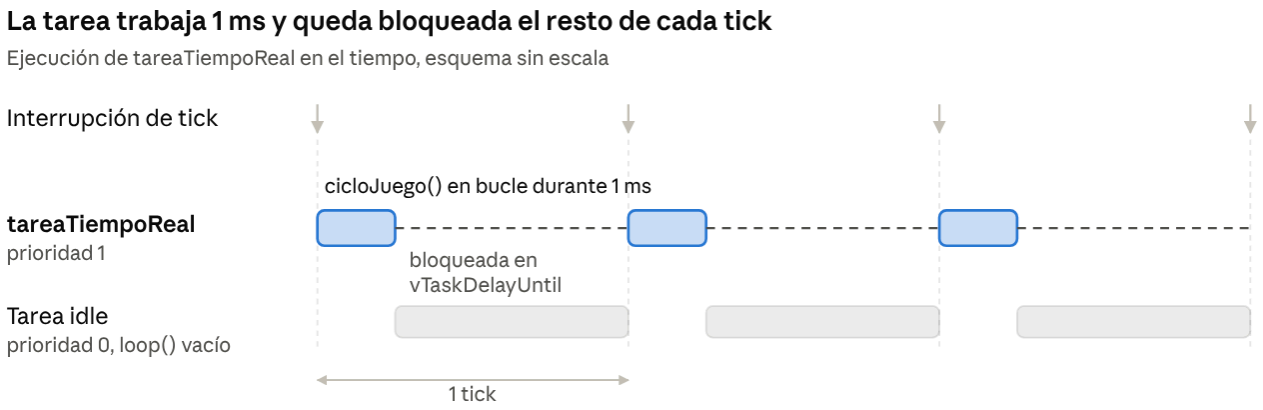
Python envía como máximo 10 mensajes por segundo a 9600 baudios. El firmware espera el formato `X:x,y,ancho,alto` y descarta cualquier mensaje con otro número de campos, valores negativos, dimensiones mayores de 4096 o coordenadas fuera del área.

8. Uso de FreeRTOS

Usamos la biblioteca Arduino_FreeRTOS con una sola tarea periódica, `tareaTiempoReal`, que ejecuta todo el control del juego. Elegimos una tarea porque cada tarea necesita su propia pila y el Arduino UNO solo tiene 2 KB de SRAM, compartidos con las variables globales, el buffer serial y la biblioteca Servo.

8.1 Creación y planificación

- En `setup()` se crea la tarea con `xTaskCreate(tareaTiempoReal, "ARENA", 210, NULL, 1, NULL)`: pila de 210 bytes y prioridad 1, mayor que la tarea idle (0). Si la creación falla, el LED integrado parpadea como diagnóstico.
- Al terminar `setup()`, la biblioteca arranca el planificador; `loop()` queda vacío y solo corre como parte de la tarea idle.
- En cada activación, la tarea repite `cicloJuego()` durante 1 ms para atender varios pulsos de motor seguidos, y después llama a `vTaskDelayUntil(&ultimoTick, 1)`.
- `vTaskDelayUntil` hace la tarea periódica con un periodo fijo medido desde la activación anterior, no desde que terminó; así el periodo no se alarga si un ciclo tarda más. Mientras espera, la tarea está bloqueada y no consume CPU, en lugar de hacer espera activa.



Ejecución de la tarea FreeRTOS en cada tick · esquema

Con el tick por defecto de Arduino_FreeRTOS en AVR (unos 15 ms, generado por el watchdog), la tarea genera pulsos de motor solo una parte de cada periodo, lo que explica el movimiento lento y con jitter (sección 12.2).

8.2 Funciones de tiempo real, activación y prioridad

Dentro de la tarea, las funciones se ejecutan en un orden fijo que actúa como prioridad: primero lo que tiene plazo más corto.

Orden	Función	Activación	Plazo o temporización	Justificación del orden
1	`leerSerial()`	Cada ciclo, si hay bytes	Buffer del UART de 64 bytes	Evita perder datos de la cámara
2	Lectura del interruptor	Cada ciclo	Inmediato	Seguridad: apagar tiene prioridad sobre todo
3	`calibrar()` o `moverCarritos()`	Cada ciclo en `CALIBRANDO` o `JUGANDO`	Pulsos de 500 µs (calibración) y 50 µs (juego)	Plazo más corto: el portero debe llegar antes que la pelota
4	`moverGolpeadores()`	Cada ciclo en `JUGANDO`	Golpe de 170 ms, mínimo 500 ms entre golpes	Depende de la posición del portero

Orden	Función	Activación	Plazo o temporización	Justificación del orden
5	`actualizarSensoresGol()`	Cada ciclo en `JUGANDO`	Confirmación de 30 ms, rearme de 300 ms	El gol tolera unos milisegundos de retraso
6	`celebrar()`	Cada ciclo en `FINALIZADO`	Cambio cada 200 ms	No es crítico

8.3 Sincronización y condiciones de carrera

Como una sola tarea lee y escribe todas las variables, no hay condiciones de carrera ni inversión de prioridades entre tareas, y por eso no se necesitan semáforos ni mutex. El único recurso compartido es el puerto serial, que solo usa esta tarea.

8.4 Arquitectura multitarea propuesta

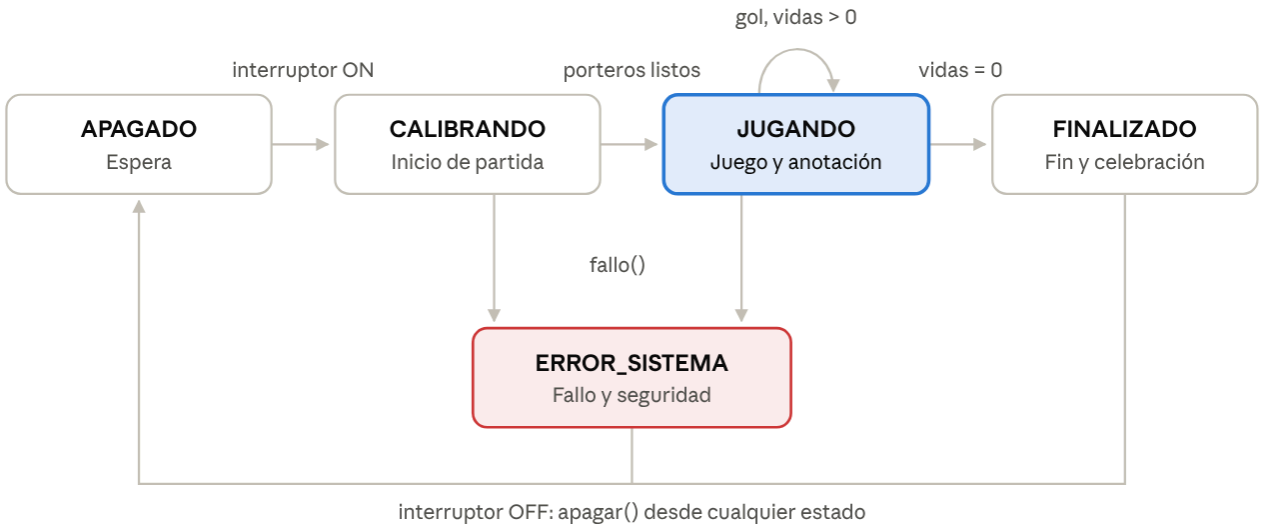
Con un microcontrolador con más memoria, el diseño recomendado separa las funciones en tareas con planificación pre-emptive:

Tarea	Prioridad	Activación	Comunicación
Motores (pulsos por timer de hardware)	Alta	Periódica, cada pocos ms	Lee el objetivo de una cola
Golpeo	Alta	Evento: pelota en zona	Recibe la orden por notificación de tarea
Percepción serial	Media	Evento: llegada de bytes	Envía coordenadas por una cola
Goles y vidas	Media	Interrupción del sensor	Semáforo binario desde la ISR
Máquina de estados	Media	Eventos de las demás tareas	Cola de eventos
LEDs y diagnóstico	Baja	Periódica, cada 100 ms	Mutex sobre el puerto serial

9. Máquina de estados del juego

La función `cicloJuego()` implementa cinco estados que cubren los siete que pide el enunciado; la anotación se procesa dentro de `JUGANDO` y la inicialización de hardware ocurre en `setup()`.

Cada gol resta una vida y con cero vidas la partida termina



Máquina de estados de cicloJuego() · 5 estados

Apagar el interruptor desde cualquier estado detiene motores y servos y apaga los LEDs; encenderlo de nuevo inicia una partida nueva con 10 vidas, sin reprogramar.

Estado del código	Estado del enunciado	Qué hace	Sale cuando
`setup()`	INICIALIZACIÓN	Configura pines, servos, LEDs y crea la tarea FreeRTOS	Termina la configuración
`APAGADO`	ESPERA	Todo detenido, LEDs apagados	Interruptor en ON
`CALIBRANDO`	INICIO DE PARTIDA	Vidas en 10, calibra ambos porteros y los centra	Ambos porteros quedan listos
`JUGANDO`	JUEGO y ANOTACIÓN	Seguimiento, golpes y detección de goles; cada gol resta una vida	Un jugador llega a 0 vidas, o hay una falla

Estado del código	Estado del enunciado	Qué hace	Sale cuando
`FINALIZADO`	FIN	Detiene actuadores, anuncia ganador y hace parpadear sus LEDs 12 veces	Interruptor en OFF
`ERROR_SISTEMA`	FALLO/SEGURIDAD	Detiene motores y servos e indica la causa por serial	Interruptor en OFF y revisión

Causas de falla detectadas por `fallo()`: dos finales activos a la vez, calibración de más de 30 s, más de 10 000 pasos sin encontrar un extremo, recorrido menor de 100 pasos, final izquierdo activado al centrar, final activado durante el juego y posición fuera del recorrido.

10. Algoritmo de percepción, predicción e intercepción

El controlador actual es de seguimiento directo: cada portero copia la posición horizontal de la pelota, escalada a su recorrido, y golpea cuando la pelota entra en su franja de la cancha. No estima velocidad ni trayectoria todavía; eso queda como la mejora principal (10.4).

10.1 Percepción (Python + OpenCV)

1. **Captura.** El video llega desde un celular por la red (DroidCam, puerto 4747).
2. **Región de interés.** Al iniciar, el usuario selecciona el campo con el ratón (`selectROI`), y desde entonces solo se procesa esa zona; esto define el sistema de coordenadas.
3. **Segmentación por color.** La imagen se pasa a HSV y se filtran dos rangos de rojo, porque el rojo está en ambos extremos del matiz: H 0–12 y H 170–179, con S ≥ 100 y V ≥ 80.
4. **Limpieza.** Apertura y cierre morfológicos con un núcleo de 3×3 eliminan ruido y rellenan huecos.
5. **Validación.** De los contornos, se toma el más grande que tenga área entre 80 y 2500 píxeles y circularidad de al menos 0.65, con `circularidad = 4π · área / perímetro²` (1 es un círculo perfecto). Esto descarta objetos rojos que no son la pelota.
6. **Envío.** El centro del círculo mínimo que encierra el contorno se manda al Arduino, como máximo cada 100 ms (10 Hz).

10.2 Posición objetivo del portero

La X de la pelota se convierte linealmente a pasos dentro del recorrido seguro, que excluye 80 pasos en cada extremo:

```
\text{objetivo} = m + \frac{x \cdot (R - 2m)}{W}
```

Donde x es la posición de la pelota en píxeles, W el ancho de la región de interés, R el recorrido total medido en la calibración y m = 80 pasos de margen. El portero se mueve un paso a la vez hacia el objetivo y se detiene cuando la diferencia es de 12 pasos o menos, lo que evita oscilaciones alrededor del punto.

10.3 Decisión de golpe

La cancha se divide verticalmente: la pelota está en la zona del portero 1 si su Y está en el 30 % superior de la imagen, y en la del portero 2 si está en el 30 % inferior. El servo golpea cuando se cumplen tres condiciones a la vez: la pelota acaba de entrar a la zona, pasaron al menos 500 ms desde el último golpe y el portero está alineado. Golpea una sola vez por cada entrada a la zona y se rearma cuando la pelota sale.

Manejo de situaciones anómalas. Si no llega un dato válido en 600 ms, el portero se detiene y se rearman los golpes; los mensajes mal formados se descartan; un final de carrera activado durante el juego o una posición fuera del recorrido llevan al estado de falla.

10.4 Mejora propuesta: predicción de trayectoria

Con dos muestras sucesivas se puede estimar la velocidad, $vx = (x_2 - x_1) / \Delta t$ y $vy = (y_2 - y_1) / \Delta t$, y calcular dónde cruzará la pelota la línea del portero. Si la pelota avanza hacia la portería (vy en su dirección), el tiempo de llegada es $t = (y_portero - y) / vy$ y la intercepción es $x + vx \cdot t$, reflejando el valor en las paredes laterales para considerar rebotes. Así el portero se adelanta en lugar de reaccionar tarde.

11. Plan y resultados de pruebas

Seguimos el orden que pide el enunciado: cada componente se prueba por separado antes de integrarlo, y luego se valida el sistema completo.

ID	Prueba	Procedimiento	Criterio de aceptación
P-01	Motor y driver individual	Mover cada NEMA 17 en ambos sentidos	Gira en los dos sentidos sin perder pasos

ID	Prueba	Procedimiento	Criterio de aceptación
P-02	Finales de carrera	Presionar cada uno y leer el monitor serial	Se lee HIGH al presionar
P-03	Calibración	Encender con el portero en distintas posiciones	Encuentra ambos extremos y queda en el centro
P-04	Falla de calibración	Bloquear un final o desconectar un motor	Se detiene con mensaje de error tras 30 s o 10 000 pasos
P-05	Servos	Ordenar un golpe a cada servo	Va al ángulo de golpe y regresa en 170 ms
P-06	Detección de la pelota	Poner la pelota en 5 posiciones y moverla a varias velocidades	El círculo verde la sigue sin falsos positivos
P-07	Comunicación serial	Enviar coordenadas y revisar la respuesta	El Arduino acepta y responde por serial
P-08	Seguimiento	Mover la pelota a lo largo de la portería	El portero se alinea dentro de 12 pasos
P-09	Pérdida de la pelota	Tapar la cámara durante el juego	El portero se detiene en menos de 600 ms
P-10	Sensores de gol	Pasar la pelota 10 veces por cada portería	10 goles contados, ninguno doble
P-11	Marcador	Anotar hasta llegar a 0	Los LEDs bajan de 10 a 0 y se anuncia el ganador
P-12	Reinicio	Apagar y encender el interruptor tras una partida	Nueva partida con 10 vidas sin reprogramar
P-13	Partida automática completa	Jugar sin intervención humana	Termina con un ganador declarado

11.1 Mediciones de tiempo real

Los tiempos de respuesta de los eventos críticos se miden con `micros()` y el monitor serial, comparándolos con los valores de diseño del código:

Evento	Cómo medirlo	Valor de diseño
Periodo de la tarea FreeRTOS	Diferencia de `micros()` entre activaciones	1 tick
Latencia cámara → Arduino	Marca de tiempo en Python vs recepción en Arduino	⩽ 100 ms entre envíos
Detección → registro de gol	Tiempo entre flanco del sensor y mensaje "GOL"	30 ms de confirmación
Orden → golpe completo	Tiempo entre "GOLPE ACTIVADO" y "SERVO REGRESO"	170 ms
Jitter del ciclo	Variación del periodo en 100 activaciones	Lo menor posible

12. Registro de retos, problemas y decisiones técnicas

Resolvimos los retos de hardware (motores, pines, calibración) y quedaron abiertos los de integración y temporización, que explican por qué el seguimiento autónomo no es todavía fluido.

12.1 Retos resueltos

Reto	Causa	Solución
Los NEMA 17 no respondían de forma correcta	Conexiones, nivel de dirección y ancho de pulso sin ajustar	Pruebas individuales de cada motor; dirección comprobada por portero (`DERECHA1` , `DERECHA2`) y pausa tras cambiar de sentido
Desconocer la posición del portero	Los motores paso a paso no tienen encoder	Calibración con finales de carrera y conteo de pasos
Choques contra los extremos	Errores de conteo o de objetivo	Margen de 80 pasos por lado y paro inmediato al activar un final
Faltaban pines para 20 LEDs	El UNO tiene 20 pines de E/S en total	Registros 74HC595 en cascada controlados con 3 pines

Reto	Causa	Solución
El Arduino no puede procesar video	16 MHz y 2 KB de RAM	Visión con OpenCV en la computadora; el Arduino solo controla
Conteos dobles de gol	La pelota activa el sensor varias veces al entrar	Confirmación de 30 ms y rearme tras 300 ms libre
Golpes repetidos	El servo se activaba en cada ciclo dentro de la zona	Un golpe por entrada a la zona y 500 ms mínimo entre golpes
Falta de memoria con varias tareas FreeRTOS	Cada tarea necesita su propia pila y el UNO solo tiene 2 KB de SRAM	Una tarea con pila de 210 bytes; las funciones se ejecutan como métodos no bloqueantes

12.2 Problemas no resueltos y su causa

Al revisar el repositorio encontramos causas concretas de los problemas de funcionamiento:

Problema	Causa	Corrección propuesta
El Arduino ignora las coordenadas de la cámara	Python envía `X:x,ancho` (2 valores) y el firmware exige `X:x,y,ancho,alto` (4); `procesarLinea()` descarta todo mensaje distinto	Enviar también Y y el alto: `f"X:{x},{y},{ancho_roi},{alto_roi}\n"`
Error ocasional en Python al leer la respuesta	La variable `respuesta` se usa aunque no se haya recibido nada	Inicializar `respuesta = ""` y mover el `if` dentro del `while`
Movimiento lento y con jitter	La tarea genera pulsos durante 1 ms y luego duerme hasta el siguiente tick; con el tick por defecto de Arduino_FreeRTOS en AVR (unos 15 ms, basado en el watchdog) el motor solo recibe pulsos una fracción del tiempo	Generar los pulsos con un timer por hardware o una interrupción, y dejar a FreeRTOS la lógica
Reacción tardía al cambiar de sentido	`PAUSA_DIRECCION` detiene el motor 500 ms en cada cambio de dirección	Reducirla a pocos milisegundos; el TB6600 necesita microsegundos

Problema	Causa	Corrección propuesta
La condición de alineación nunca bloquea un golpe	`MARGEN_ALINEACION` vale 10 000 pasos, más que el recorrido máximo	Bajarlo a un valor cercano al ancho del portero en pasos
Sensibilidad a la iluminación	La detección depende de un rango fijo de color rojo	Calibrar el rango HSV en el lugar de la demostración o usar luz controlada
Sin predicción de trayectoria	El portero sigue la posición actual, no la futura	Algoritmo de la sección 10.4

12.3 Decisiones técnicas

- Cartón en lugar de acrílico o MDF, por costo y facilidad de modificar.
- Servos en lugar de solenoide, para evitar la etapa de potencia de 12 V con MOSFET y diodo.
- Visión en la computadora en lugar de sensores en la arena, para obtener X y Y con un solo sensor.
- Una tarea FreeRTOS en lugar de varias, por la memoria del UNO.
- Reinicio con el interruptor en lugar de un botón dedicado, para no ocupar otro pin.

13. Competencias adquiridas y conclusiones

El proyecto integró construcción, electrónica, visión artificial y programación de tiempo real en un solo sistema, y nos enseñó que en tiempo real el límite casi nunca es la lógica sino la temporización y los recursos del hardware.

Competencias técnicas fortalecidas:

- **Sistemas de tiempo real:** planificación periódica con FreeRTOS (`xTaskCreate` , `vTaskDelayUntil`), efecto del tick en el jitter y presupuesto de memoria por tarea.
- **Programación embebida no bloqueante:** máquinas de estado con `micros()` y `millis()` en lugar de `delay()` .
- **Control de motores:** drivers TB6600, generación de pulsos, calibración por finales de carrera y conteo de pasos.
- **Visión artificial:** segmentación HSV, operaciones morfológicas y filtrado por área y circularidad con OpenCV.
- **Comunicación serial:** diseño y validación de un protocolo de mensajes entre Python y Arduino.

- **Electrónica:** separación de fuentes de potencia y lógica, registros de desplazamiento y entradas con pull-up.
- **Trabajo en equipo e ingeniería:** pruebas por componente antes de integrar y documentación de decisiones.

Conclusiones:

1. La división entre percepción en la computadora y control en el Arduino fue acertada: cada parte trabaja con la exigencia de tiempo que puede cumplir.
2. El Arduino UNO limita el uso de FreeRTOS: con 2 KB de SRAM no caben varias tareas, y el tick del planificador es demasiado largo para generar pulsos de motor de microsegundos.
3. Los problemas pendientes tienen causas identificadas y correcciones concretas (sección 12.2); la más urgente es igualar el formato de mensaje entre Python y el firmware.
4. Como trabajo futuro: predicción de trayectoria, migrar a un microcontrolador con más memoria (por ejemplo, ESP32) para dividir el sistema en varias tareas con prioridades, colas y mutex, y medir formalmente los tiempos de respuesta.